

Day 19: ANOVA Python Implementation

MODULE: INFERENCEAL STATISTICS • MANUAL BLUEPRINT & SCIPY POSTMORTEM

1. Pure Manual Python Implementation

Objective: Writing custom Python scripts from scratch without relying on advanced statistical packages to replicate the core mathematical formulas of ANOVA (*MSB*, *MSW*, and F-Score calculation).

The objective of writing this manual code is to implement the underlying mathematical logic step-by-step. Instead of relying on high-level library shortcuts, we programmatically compute individual variance components, evaluate the differences between group means scaled by sample sizes, and incorporate systemic validation checks.

```

# Data Setup for Artisans (Abdullah, Badshah, Vishal)
abdullah = [10, 11, 9]
badshah = [12, 13, 11]
vishal = [8, 9, 7]
all_data = abdullah + badshah + vishal
N = len(all_data) # Total items = 9
k = 3 # Total groups = 3

# Calculating Group Means and Grand Mean
m_abdullah = sum(abdullah) / len(abdullah) # Mean = 10.0
m_badshah = sum(badshah) / len(badshah) # Mean = 12.0
m_vishal = sum(vishal) / len(vishal) # Mean = 8.0
grand_mean = sum(all_data) / N # Grand Mean = 10.0

# MSW Calculation (Variance Within Groups)
ssw = sum([(x - m_abdullah)**2 for x in abdullah]) + sum([(x - m_badshah)**2 for x in badshah]) + sum([(x - m_vishal)**2 for x in vishal])
df2 = N - k
msw = ssw / df2 # Output: 6 / 6 = 1.0

# MSB Calculation (Variance Between Groups with Sample Weight Multiplication)
n_pieces = 3
ssb = n_pieces * ((m_abdullah - grand_mean)**2 + (m_badshah - grand_mean)**2 + (m_vishal - grand_mean)**2)
df1 = k - 1
msb = ssb / df1 # Output: 24 / 2 = 12.0

# Final F-Score Ratio & Decision Block
f_score = msb / msw # Output: 12.0 / 1.0 = 12.0

if f_score == 1.0 and msw > 5.0:
    print("🚨 SYSTEM RED ALERT: High Systemic Issue!")
elif f_score > 4.0:
    print("❌ NULL HYPOTHESIS REJECTED: Significant structural variance detected!")
else:
    print("✅ NULL HYPOTHESIS WINS: Variations fall within standard boundaries.")

```

2. Automated Execution via SciPy Library

Objective: Utilizing optimized distribution functions from the SciPy ecosystem to significantly streamline code execution while simultaneously retrieving both the F-Statistic and its corresponding P-Value.

In modern analytical and production pipelines, manual calculations are typically bypassed in favor of the SciPy framework. This allows single-line execution to extract precise metrics followed by an automated conditional block for statistical hypothesis evaluation.

```
from scipy.stats import f_oneway

# Single-Line Execution
f_score, p_value = f_oneway(abdullah, badshah, vishal)

print(f"F-Score: {f_score:.2f}, P-Value: {p_value:.4f}")

# Automated Decision Evaluation Block
if p_value < 0.05:
    print("❌ NULL HYPOTHESIS REJECTED: Statistically significant difference observed!")
else:
    print("✅ NULL HYPOTHESIS WINS: Variation is due to random chance.")
```

3. Structural Postmortem of the 'f_oneway' Function

Technical Breakdown: Examining the internal routines of the `f_oneway` function, which processes independent arrays to isolate categorical variance against residual noise.

A deep architectural review of this function highlights its reliance on the SciPy core library framework. It is designated as `f_oneway` because it performs a One-Way Analysis of Variance, meaning it isolates variations driven exclusively by a single controlled categorical factor (e.g., the change in Artisan).

The routine processes the inputs synchronously to return two critical statistical variables:

1. **f_score:** A ratio determining how many times the variance between the groups outweighs the variance within the groups (yielding an exact factor of 12.0 in this dataset).
2. **p_value:** The exact mathematical probability that the observed differences could occur under a true null hypothesis. The resulting value of 0.0080 indicates a less than 1% probability of random occurrence, validating the complete rejection of the Null Hypothesis.